AI Application Engineer Interview Bank

Last curated: 8 August 2026

Answers are deliberately concise. Practice a 30-second answer, then extend with the financial-document project and a failure/measurement.

Project deep dive

1. What have you actually built?

I built a typed financial extraction prototype with Pydantic models, a direct Gemini structured-output call, fake malformed/fabricated scenarios, a verbatim evidence check, tests, and basic chunk/prompt utilities. Course labs cover LangChain/LlamaIndex RAG, but the main package does not yet have vector retrieval, FastAPI, agents, or deployment. I describe those as the measured roadmap, not completed features.

2. What is the most important limitation you discovered?

A Pydantic-valid response can still be false. Even a quote that occurs in the source can be paired with the wrong metric, value, unit, company, or period. The next validation layer must check claim-to-evidence consistency and evaluate representative adversarial cases.

3. How would you turn this prototype into job-ready evidence?

Add stable ingestion/provenance, measured retrieval, hybrid search/reranking only if it improves evals, citation support checks, a versioned eval set, FastAPI boundaries, reliability controls, traces, threat tests, and a Dockerized deployment. Publish before/after metrics and one diagnosed failure.

4. Why did you use `Decimal` for financial values?

Binary floating-point can introduce representation/rounding errors. `Decimal` supports explicit decimal precision and rounding, which is more appropriate for financial values. The model should extract values; deterministic code should perform arithmetic and validation.

LLM boundaries and prompting

5. What does structured output solve?

It reduces malformed-output handling by constraining response shape and types. It does not verify factuality, source support, authorization, or safety.

6. Is temperature zero deterministic?

It usually reduces sampling variability, but it is not a universal reproducibility guarantee. Provider/model updates, infrastructure, floating-point effects, and tie-breaking can change output. Use evals and version tracking rather than relying on temperature.

7. When would you use few-shot examples?

When examples measurably clarify output structure, edge cases, or classification boundaries. They increase tokens and can bias behavior, so compare against a zero-shot baseline on representative cases.

8. Prompting, long context, RAG, fine-tuning, or tools?

Prompting supplies instructions; long context supplies a small request-scoped corpus; RAG selects from large/dynamic/private knowledge; fine-tuning changes learned behavior; tools query authoritative systems, compute, or act. Choose based on the missing capability and evaluate the simplest option.

RAG system design

9. Explain RAG end to end.

Offline: authorize sources, parse, normalize, deduplicate, chunk with provenance, embed, index, and verify. Online: authenticate, filter, retrieve, fuse/rerank if useful, budget context, generate/refuse, cite, validate support, and trace the request.

10. A RAG answer is wrong. How do you debug it?

Inspect the source parse, expected chunk, index/metadata, retrieved ranks/scores, access filters, context assembly, prompt/model output, and citation validation in order. Retrieval metrics distinguish a retrieval miss from a generation failure.

11. How do you choose chunk size and overlap?

Start with document structure and an evaluation baseline. Compare configurations on retrieval recall/precision, answer/citation quality, latency, and cost. Overlap can preserve continuity but increases duplication and storage.

12. Dense versus sparse versus hybrid search?

Dense retrieval captures semantic similarity. Sparse/BM25 captures exact terms, identifiers, rare names, and figures. Hybrid retrieval combines both; financial/technical corpora often benefit, but the improvement should be measured.

13. What does a reranker do?

It jointly scores a query and each candidate, often improving final ordering beyond independent embeddings. It adds latency/cost, so compare retrieval and answer metrics before and after.

14. Explain recall@k and MRR.

Recall@k asks whether relevant evidence appears in the first k results. Mean reciprocal rank rewards placing the first relevant result earlier. Neither proves the generator used the evidence correctly.

15. What is faithfulness?

Whether answer claims are supported by the supplied context. It is separate from answer relevance and correctness: an answer can be relevant yet unsupported, or faithful to context that is itself wrong.

16. Why are returned source documents not enough?

They show what retrieval selected, not which claim they support. The model may ignore them or make an unsupported statement. Use stable citations and claim-to-span support checks.

17. How do metadata filters affect security?

They enforce tenant, user, document, region, or time scope before candidates reach the model. A missing/incorrect filter can leak data even if the final answer prompt says not to.

18. When would you use long context instead of RAG?

When the authorized corpus is small, fits comfortably within the context/token budget, and retrieval infrastructure adds more complexity than value. Evaluate cost, latency, and lost-in-the-middle behavior.

Evaluation

19. How would you build a RAG eval set?

Version questions, expected claims/answers, relevant source IDs, expected refusals, and tags. Include happy paths, conflicts, missing facts, exact IDs, units/periods, injection, duplicates, access filters, and provider failures.

20. Which metrics belong in a release gate?

Retrieval recall/precision, answer/citation correctness, refusal accuracy, schema/tool success, safety cases, latency percentiles, error rate, and cost. Choose thresholds from use-case risk and prevent unacceptable regressions rather than chasing one aggregate score.

21. What are LLM-as-judge limitations?

Bias, variance, prompt sensitivity, position/style preference, self-preference, and model-version drift. Calibrate against human labels, use clear rubrics/multiple checks where appropriate, and retain deterministic graders for exact facts.

22. How do evals differ from production monitoring?

Offline evals compare controlled, labeled cases before release. Production monitoring detects real distribution, latency, cost, safety, and operational failures. Production samples can become reviewed future eval cases.

FastAPI and reliability

23. How would you structure the service?

Thin typed FastAPI handlers call domain services; provider/storage/retrieval adapters sit behind interfaces. Long ingestion runs outside the request path. Tests mock network providers and integration tests cover boundaries.

24. When does async help?

For concurrent I/O such as model/database calls when libraries are async and concurrency is bounded. It does not make CPU-bound parsing faster; use workers/processes or background jobs for that.

25. How do you handle model-provider failures?

Explicit timeouts, bounded retries with backoff/jitter only for retryable errors, concurrency/rate limits, cancellation, error mapping, fallbacks where justified, and observability. Avoid retrying non-idempotent actions without idempotency controls.

26. When would you stream responses?

When lower perceived latency is valuable. Streaming complicates validation, moderation, error semantics, cancellation, tracing, and clients, so do not make it the default without a product reason.

27. Why background ingestion?

Parsing, embedding, and indexing can exceed request timeouts and need retries/resume. Return a job ID, make ingestion idempotent, persist status, bound the queue, and expose progress/failure.

Tools, agents, LangGraph, and MCP

28. Chain, workflow, or agent?

A chain has fixed composition; a workflow has application-controlled state/branches; an agent lets the model dynamically choose constrained actions. Prefer the least dynamic design that meets the requirement.

29. How do you make tool calls safe?

Authenticate/authorize outside the model, validate typed arguments, enforce allowlists/resource scope, limit time/steps/tokens, handle retries/idempotency, sanitize outputs, audit calls, and require approval for consequential actions.

30. How do you stop runaway agents?

Define terminal states and budgets for iterations, time, tokens, and tool calls. Add cancellation, repeated-action detection, failure routes, checkpoints/approval where useful, and regression tests for loops.

31. When is LangGraph justified?

When explicit state, branching, cycles, checkpointing, recovery, or human interrupts improve control. Compare with direct Python; do not add a graph to a simple fixed pipeline.

32. What does MCP provide?

It standardizes discovery and invocation of tools/resources/prompts across hosts and servers. It does not automatically provide trust, least privilege, authorization, or prompt-injection protection.

33. Name MCP security concerns.

Token theft/passthrough, confused-deputy attacks, malicious metadata, excessive scopes, local-server compromise, SSRF, exfiltration, and indirect prompt injection. Use least privilege, consent, audience validation, secure token handling, and application-side policy.

Observability, security, and operations

34. What would you trace?

Correlated spans for request handling, retrieval, reranking, model and tool calls, validation, and storage. Record safe identifiers, timings, status, model/token/cost data, ranks, retries, and denied actions without leaking secrets or sensitive content.

35. How do you defend against indirect prompt injection?

Treat retrieved/tool content as untrusted, delimit it, prevent it from changing system policy, keep authorization outside the model, minimize tools/scopes, require approval for side effects, validate outputs, and maintain adversarial evals. No single prompt solves it.

36. What are the biggest RAG/agent cost controls?

Context budgets, retrieval/reranking limits, model selection based on evals, concurrency/rate limits, bounded retries and agent steps, batch/background work, measured caching, and per-request/tenant budgets with alerts.

37. What can go wrong with semantic caching?

False semantic matches, stale answers, cross-tenant leakage, ignored authorization/context, and hard invalidation. Use tenant-aware keys, TTL/versioning, conservative thresholds, sensitive/dynamic-query bypasses, and evaluation.

38. What is the value of NIST/OWASP guidance?

They provide structured risk categories and control prompts for governance, measurement, security, privacy, and operations. Map relevant threats to controls/tests/owners rather than treating a checklist as proof of safety.

39. What should a production-ready README prove?

Exact implemented status, architecture/data flow, reproducible setup, API examples, tests/eval methodology and results, security boundaries, observability, deployment, cost assumptions, known limitations, and roadmap.

40. What makes this portfolio credible to a hiring manager?

One deployed system with honest scope, measured retrieval/generation quality, safe boundaries, failure handling, tests/traces, and a clear design narrative is stronger than many unmeasured framework demos.

System-design prompt

Design a multi-tenant earnings-document assistant. Cover ingestion, provenance, access controls, parsing, hybrid retrieval, reranking, citations/refusal, evaluation, tool authorization, FastAPI/background work, storage, observability, deployment, cost, threat model, rollout, and incident response. State assumptions and defend every added component.